nanoGenRec: A Reproducible Framework and Empirical Study for Semantic-ID Generative Recommendation

Author Name
Affiliation
email@example.com

June 7, 2026

Abstract

Generative recommendation has recently reframed large-scale retrieval as autoregressive generation over Semantic IDs, with industrial systems reporting gains from model scaling, reinforcement learning, and end-to-end deployment. However, reproducing this line of work remains difficult because the complete pipeline spans item tokenization, behavior sequence construction, next-token prediction, constrained generation, full-recall evaluation, experiment orchestration, and long-running operational logs. We present nanoGenRec, an open-source framework for Semantic-ID generative recommendation, together with a production-scale empirical study on anonymized behavior data. The framework implements Semantic ID tokenization, Transformer+MoE next-token recommendation, SID-constrained beam-search evaluation, YAML-based experiment configuration, duplicate-run detection, queue-based long jobs, and durable experiment logs. Across 51 recorded experiments, we report five empirical findings: (i) NTP loss follows a model-scaling curve $L(N) = 2.522 + 2055.1/N^{0.456}$ from 1.7M to 101.1M active parameters; (ii) behavior-window scaling is non-monotonic under production temporal drift; (iii) tokenizer capacity and embedding scale jointly determine collision, semantic-neighbor retention, and downstream recall; (iv) full-recall evaluation reaches R@500=70.4% and R@10=14.2% for an M-tier 4B-SID model; and (v) post-training alignment is sensitive to candidate distribution, with off-policy ECPO collapsing to R@500=2.0% while on-policy candidates recover to 67.8%. We position nanoGenRec as a reproducibility artifact and empirical reference for researchers and engineers building Generative Recommendation systems.

1 Introduction

Generative Recommendation (GR) is moving recommender systems from multi-stage retrieve-and-rank pipelines toward autoregressive generation over item tokens. Industrial reports such as OneRec [5], OneRec-V2 [6], OneMall [4], GenRec [7], OneLive [2], and GR4AD [3] show that Semantic-ID generation, model scaling, and preference alignment can be viable at production scale.

The public literature now contains strong system diagrams and deployment evidence, yet there is still a practical gap for researchers who want to reproduce the full workflow. A working GR system needs more than a model definition. It needs a tokenizer that maps items into discrete semantic tokens, a behavior preprocessing path that produces autoregressive training sequences, a model and training loop, a constrained generation procedure that maps generated tokens back to items, a full-recall evaluation protocol, and experiment infrastructure that can distinguish real improvements from duplicated baselines, invalid feature plumbing, or inline-evaluation artifacts.

We introduce nanoGenRec, an open-source framework and empirical record for Semantic-ID-based generative recommendation. The project is built from production-grounded experiments,

with private data and deployment-specific details removed. Its purpose is not to claim a new recommendation algorithm. Instead, it provides a reproducible framework, a full evaluation protocol, and an experiment lineage that records both positive results and failure modes.

This report makes four contributions:

1. **Framework.** We describe an end-to-end GR workspace covering Semantic ID tokenization, NTP training, SID-constrained beam search, full-recall evaluation, YAML experiment configuration, duplicate-run checks, and queue-based long jobs.

2. **Empirical scaling.** We report a seven-point NTP model-scaling sweep from 1.7M to 101.1M active parameters and fit a power law with exponent 0.456, close to the OneRec-V2 reported exponent of 0.489.

3. **Production data observations.** We show that behavior-window scaling is non-monotonic in a production setting because recency, user breadth, and per-user sequence depth interact under temporal drift.

4. **Failure-aware reproducibility.** We preserve experiment logs for successes, negative results, invalid runs, and post-training collapses, including an off-policy ECPO failure and an on-policy recovery.

2 Background

2.1 Semantic-ID Generative Recommendation

Semantic-ID GR represents each item as a short sequence of discrete tokens. Recommendation then becomes conditional generation: given a user's behavior history, a model predicts the token sequence of the next target item. This formulation allows recommendation to reuse autoregressive modeling, beam search, MoE scaling, and post-training alignment ideas from generative AI.

Recent industrial systems have explored this direction from complementary angles. OneRec introduced an end-to-end generative architecture and reported production deployment with scaling and reinforcement learning evidence [5]. OneRec-V2 improved scalability with a lazy decoder-only design and real-world preference alignment [6]. OneMall unified several e-commerce scenarios with Semantic Tokenization, Transformer-based generation, and RL alignment [4]. GenRec introduced page-wise NTP, token merging, and GRPO-SR for preference-oriented recommendation [7]. OneLive adapted the paradigm to live-streaming with dynamic tokenization and multi-objective alignment [2]. GR4AD studied production advertising with value-aware learning and dynamic beam serving [3].

2.2 Reproducibility Gap

GR papers typically report architecture and production results, while the operational details needed for reproduction are scattered across preprocessing, training, evaluation, and experiment management. Several details are easy to miss:

- Inline evaluation during training is often a health check, not a comparable full-recall metric.
- Feature ablations can become invalid when preprocessing stores a feature but training or inference does not consume it.

- Tokenizer proxy metrics can disagree unless collision, balance, and behavior-aware semantic-neighbor retention are tracked together.
- RL-style post-training can collapse when rollout candidates are sampled from a distribution that no longer matches the current policy.

nanoGenRec treats these details as first-class experimental artifacts.

3 The nanoGenRec Framework

3.1 Pipeline

The framework consists of six stages:

1. **Item representation.** Items are embedded with pretrained representation models. The current experiments use Qwen3 embeddings at several scales.
2. **Semantic ID tokenization.** Embeddings are discretized into 3-token item IDs using residual KMeans and FSQ-family variants.
3. **NTP preprocessing.** User behavior logs are converted into autoregressive training shards, with explicit date windows and evaluation targets.
4. **Generative recommendation.** A Transformer+MoE NTP model predicts item SID tokens from user histories.
5. **Full-recall evaluation.** SID-constrained beam search generates candidate item IDs, and Recall@K is computed with $K \in \{10, 50, 100, 500\}$.
6. **Experiment governance.** YAML configs, registry hashes, duplicate-run checks, queue-based execution, and phase-level README files preserve experiment lineage.

3.2 Experiment Orchestration

New experiments inherit from a shared YAML base config and override only the parameters being tested. Multi-variant experiments are represented as a variant list. A registry hash excludes runtime paths and environment keys, allowing the runner to surface identical or near-identical historical runs before launching a new job. Queue-friendly wrappers support long-running training and evaluation without turning experiment scripts into brittle chains.

3.3 Research-Agent Workflow

The repository includes an inbox/outbox protocol, paper notes, decision records, and status files. This is a lightweight workflow for human-agent collaboration over a research line. The agent reads papers, proposes ideas, estimates runtime, writes configs, checks prior experiments, and records decisions. The contribution here is operational: the research process is made inspectable and recoverable rather than embedded in transient chat logs or shell history.

Table 1: NTP model tiers used in the empirical study.

Tier	Embed Dim	Layers	Experts	Active Params
S-tier	256	6	8	∼17.5M
M-tier	512	8	8	∼71.6M
L-tier	512	12	16	∼101.1M

Table 2: NTP model scaling on a fixed 262M-token dataset.

Config	Active Params	PPL	Loss	R@10	R@500
scale-01	1.7M	235.1	5.460	1.9%	23.6%
scale-02	3.6M	100.4	4.609	3.7%	31.7%
scale-03	5.1M	69.6	4.243	5.4%	45.6%
scale-04	17.5M	28.1	3.334	9.8%	60.5%
scale-05	34.5M	24.0	3.178	11.5%	62.5%
scale-06	71.6M	20.8	3.037	12.6%	66.2%
scale-07	101.1M	19.4	2.965	13.7%	65.8%

4 Experimental Setup

4.1 Data

The empirical study uses anonymized production behavior data. The largest behavior sweep covers up to 7.85M users and 299.0M raw interactions. After sequence truncation, this corresponds to roughly 445M effective SID tokens. Private data and deployment-specific details are not released; the open-source artifact preserves the code, configs, evaluation protocol, and experiment logs.

4.2 Model Tiers

We use three NTP model tiers, summarized in Table 1. Active parameters count only the routed compute used by each token.

4.3 Evaluation Protocol

Full evaluation uses SID-constrained beam search and reports item recall at several cutoffs. Unless otherwise stated, reported R@K values refer to full evaluation rather than inline training evaluation. This distinction matters because inline evaluation uses a limited candidate set and is intended for training health checks.

5 Empirical Study

5.1 NTP Model Scaling

We first trained seven model sizes from 1.7M to 101.1M active parameters on a fixed 262M-token dataset. Table 2 reports the results.

The fitted loss curve is:

$$L(N) = 2.522 + \frac{2055.1}{N^{0.456}}. \tag{1}$$

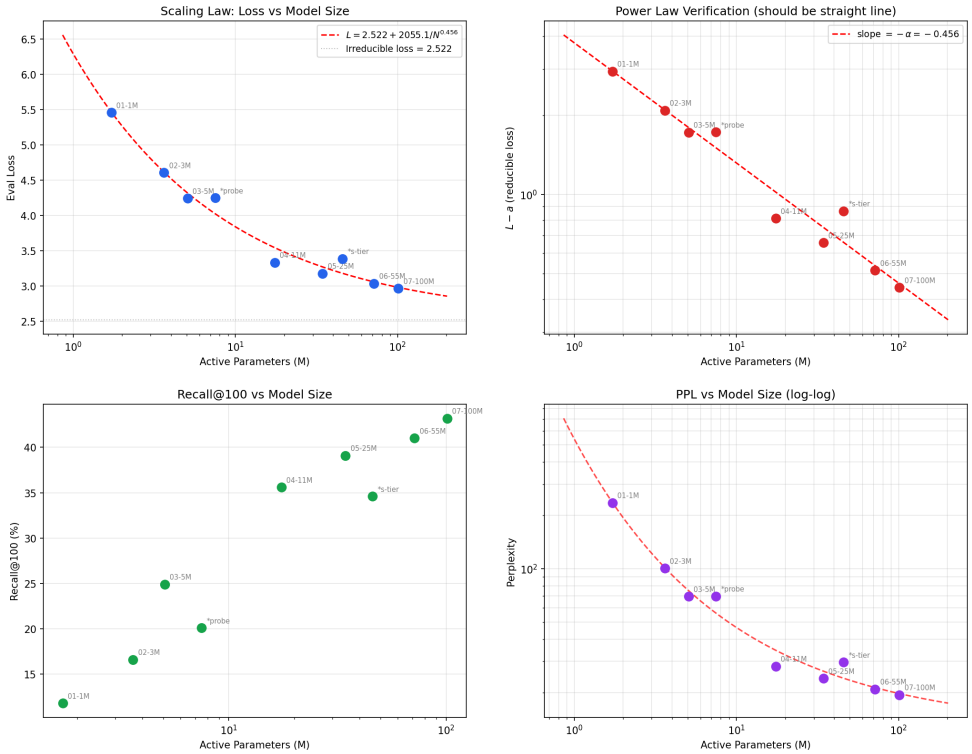

Figure 1: NTP scaling law over active parameters.

The exponent is close to the 0.489 exponent reported by OneRec-V2 [6]. Recall improves substantially through the S-tier and M-tier regime, while gains flatten between 71.6M and 101.1M active parameters under the fixed data budget. Figure 1 shows the fitted curve.

5.2 Behavior-Window Scaling

We next fixed the model tier and varied the behavior window. Standard compute-optimal scaling intuition [1] suggests that more data should reduce loss under an i.i.d. assumption. Production recommendation behavior does not satisfy this assumption. Table 3 summarizes the S-tier and M-tier data sweep.

The main observation is non-monotonicity. Extending the window increases the number of users more than the per-user sequence depth, while older behavior can drift away from the current item pool. In this dataset, the exposure window is short enough that old behavior can become stale. Figure 2 visualizes the effect.

5.3 Tokenizer and Embedding Sweep

Tokenizer quality affects the irreducible loss floor and the search space exposed to beam decoding. We evaluated 14 tokenizer variants over 0.6B and 4B embeddings, uniform 4096/8192 codebooks, MGMR-style hierarchical codebooks, and FSQ hidden sizes. Table 4 shows representative results.

The decisive variable in this sweep is the 8192 codebook setting, which improves L2 balance and snHR. The 4B embedding improves semantic-neighbor retention, but also requires enough FSQ capacity to avoid L2 entropy collapse. The current recommended caches are listed in the released experiment logs.

Table 3: Behavior-window scaling. The best loss occurs at 14 days, not at the longest window.

Config	Days	Tokens	Users	PPL	R@500
A-7d-S	7	65M	1.02M	30.60	62.1%
B-14d-S	14	130M	1.69M	27.05	58.5%
C-31d-S	31	262M	3.04M	28.05	60.5%
D-62d-S	62	441M	4.86M	30.03	58.6%
E-90d-S	90	553M	6.18M	31.89	56.2%
A-7d-M	7	65M	1.02M	19.31	70.7%
B-14d-M	14	130M	1.69M	18.96	65.8%
C-31d-M	31	262M	3.04M	19.39	65.8%
D-62d-M	62	441M	4.86M	19.80	68.1%

Table 4: Representative tokenizer sweep results. CR is collision rate and snHR is behavior-aware semantic neighbor hit rate.

SID Cache	CR	Gini-d1	Gini-d2	snHR
0.6B nc4096 h64	0.67%	0.3243	0.3546	0.0797
0.6B nc8192 h128	0.42%	0.3914	0.2375	0.0919
4B nc4096 h128	1.60%	0.2877	0.3539	0.1255
4B nc8192 h128	1.28%	0.3192	0.2530	0.1307
4B nc8192x2048 h128	1.41%	0.3191	0.3170	0.1154

5.4 Full-Recall Baselines

Table 5 reports current full-eval baselines. The strongest observed NTP result is the M-tier model with 4B SID, reaching R@500=70.4% and R@10=14.2%.

5.5 Post-Training Alignment

We also evaluated preference and RL-style alignment methods, including SP-DPO, RF-DPO, GRPO, and ECPO. The most instructive result is the off-policy/on-policy ECPO comparison in Table 6.

The off-policy run generates candidates from a reference model whose distribution drifts away from the current policy. This distorts the importance ratio and causes clipping to dominate. On-policy beam candidates better match the current policy distribution, recovering both PPL and R@500. Figure 3 shows the post-training curves.

6 Discussion

6.1 What the Artifact Makes Reproducible

The production data cannot be released. However, the artifact makes the following components inspectable and reusable:

- model and tokenizer implementations;
- preprocessing and evaluation interfaces;

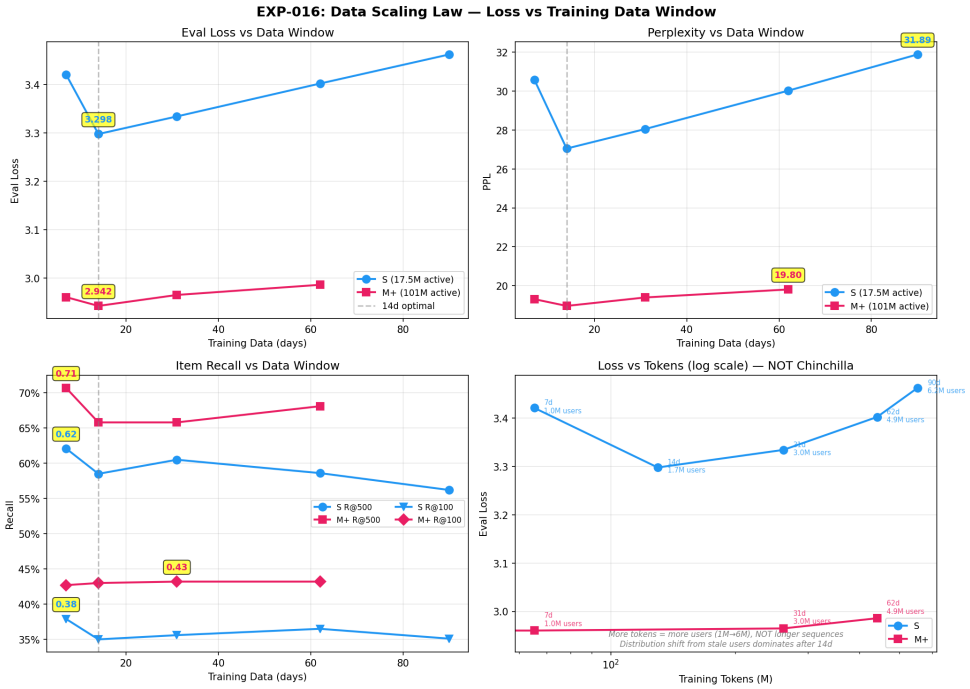

Figure 2: Behavior-window scaling under production temporal drift.

Table 5: Current full-eval baselines.

Config	R@10	R@500	PPL
S-tier bare	11.4%	61.2%	26.52
S-tier with TO-RoPE ts=0.5	11.8%	63.9%	22.7
M-tier bare, 0.6B SID	14.5%	70.2%	18.54
M-tier, 4B SID	14.2%	70.4%	16.55
L-tier with validated options	12.8%	64.1%	20.7

- experiment configs and resolved parameter sets;
- full-recall evaluation protocol;
- experiment logs with hypotheses, designs, results, and post-hoc analysis;
- known invalid runs and bug records.

This distinction matters. A framework can be reproducible even when a production dataset is not public, provided that the code path, configuration semantics, evaluation protocol, and empirical record are exposed.

6.2 Lessons

Three lessons recur across the experiment lineage. First, full evaluation must be separated from training-time health checks. Second, feature ablations require end-to-end validation across preprocessing, training, and inference; otherwise, experiments can silently compare zeros. Third,

Table 6: On-policy candidate generation recovers an ECPO collapse.

Config	R@10	R@500	PPL	Clip Rate
Off-policy ECPO	0.7%	2.0%	3791	99%
On-policy ECPO	13.0%	67.8%	14.1	92%
SFT baseline	14.1%	66.2%	16.3	–

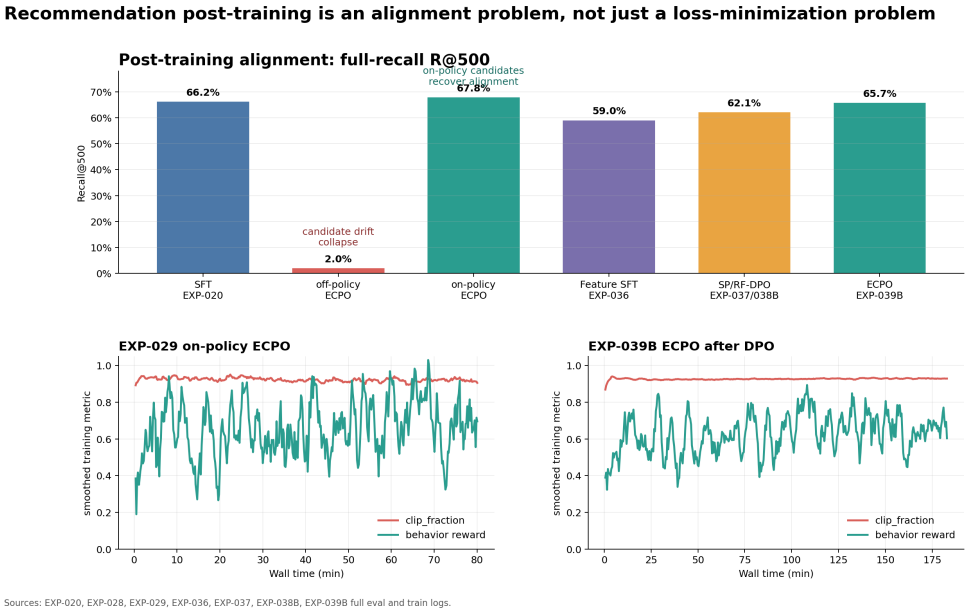

Figure 3: Post-training alignment curves and collapse/recovery behavior.

post-training alignment in GR is sensitive to candidate distribution, so rollout generation should be treated as part of the algorithm rather than a fixed data-preparation step.

7 Limitations

This report has several limitations. The production behavior data is not released, so external users cannot exactly reproduce the reported production-scale numbers. The framework currently focuses on Semantic-ID autoregressive recommendation rather than all forms of generative retrieval. Online serving, latency, and production integration are not the main focus of the artifact. Some post-training methods are preliminary and should be interpreted as engineering evidence rather than final algorithmic conclusions. Finally, public benchmark reproduction remains future work.

8 Conclusion

We presented nanoGenRec, an open-source reproducibility framework and empirical study for Semantic-ID generative recommendation. The project contributes an end-to-end framework, full-recall evaluation protocol, experiment orchestration system, and 51-experiment lineage. Empirically, it reports model scaling, non-monotonic behavior-window scaling, tokenizer sweeps, full-recall baselines, and post-training alignment failure modes. We hope the artifact helps researchers and

practitioners turn Generative Recommendation from a collection of system diagrams into an inspectable, extensible, and production-grounded research workflow.

Artifact. The code and experiment logs are available in the public nanoGenRec repository.[1]

References

[1] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, 2022. URL `https://arxiv.org/abs/2203.15556`.

[2] Shen Wang, Yusheng Huang, Ruochen Yang, et al. Onelive: Dynamically unified generative framework for live-streaming recommendation, 2026. URL `https://arxiv.org/abs/2602.08612`.

[3] Ben Xue, Dan Liu, Lixiang Wang, et al. Generative recommendation for large-scale advertising, 2026. URL `https://arxiv.org/abs/2602.22732`.

[4] Kun Zhang, Jingming Zhang, Wei Cheng, et al. Onemall: One architecture, more scenarios – end-to-end generative recommender family at kuaishou e-commerce, 2026. URL `https://arxiv.org/abs/2601.21770`.

[5] Guorui Zhou et al. Onerec technical report, 2025. URL `https://arxiv.org/abs/2506.13695`.

[6] Guorui Zhou et al. Onerec-v2 technical report, 2025. URL `https://arxiv.org/abs/2508.20900`.

[7] Yanyan Zou, Junbo Qi, Lunsong Huang, Yu Li, Kewei Xu, Jiahao Gao, Binglei Zhao, Xuanhua Yang, Sulong Xu, and Shengjie Li. Genrec: A preference-oriented generative framework for large-scale recommendation, 2026. URL `https://arxiv.org/abs/2604.14878`.

[1] `https://github.com/yzq986/nanoGenRec`